

Uncovering Layers of Vulnerability Signal from Code Changes with Machine Learning

Kiyoshi Watanabe

Abstract

This report studies vulnerability detection as a layered signal-recovery problem rather than as a single binary-classification benchmark. Using MegaVul function-pair artifacts, the project evaluates which parts of a vulnerability fix are recoverable from before-code, after-code, diff text, metadata, and engineered patch features. The experimental ladder compares function-only features, raw patch information, neutral structured diff features, security-related features, and family-motif indicators under both random and repository-disjoint evaluation. The strongest results come from structured standard-ML baselines, especially XGBoost with neutral structured diff features and family motifs. Family-level analysis shows that detectability depends on how visibly a vulnerability family appears in the local patch: input-validation, memory/bounds, and null-pointer fixes expose clearer local signals, while resource/lifetime patterns remain harder to capture. MLP and transformer extensions do not provide strong evidence of incremental value beyond the best structured XGBoost baseline. Overall, the project finds that the most recoverable signal is local, patch-structural, and representation-sensitive.

1. Problem and Study Map

Software vulnerability detection is often framed as a function-level binary classification problem: given a function, predict whether it is vulnerable [1]. This formulation is convenient, but it hides the central difficulty of this project: a single binary label compresses many different vulnerability mechanisms, while a local code artifact exposes only part of the relevant security signal. A model can therefore score well by exploiting lexical cues, repeated templates, or repository-specific artifacts without learning transferable vulnerability-related structure [1].

This project reframes the task as a **layered signal-recovery problem**. The goal is not to claim the best vulnerability detector. The goal is to ask which representation layers make vulnerability-fix signal visible, and whether that signal transfers under a stricter repository-disjoint split.

Figure 1 summarizes the study design at a high level. The left side defines the unit of learning: one patch-level before/after function-pair object. The middle stage represents feature extraction, where that object is converted into text, numeric, and motif-based feature blocks. The right side shows the model comparison stage, where the same representation ladder is evaluated using Logistic Regression, XGBoost, MLP, and Transformer baselines. The formal definition of the learnable object is given in Section 3.

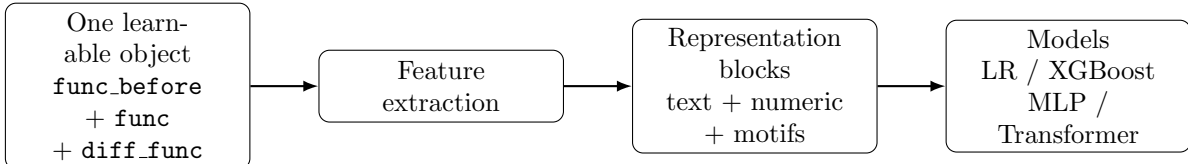


Figure 1: Study map. One patch-level function-pair object is converted into representation blocks and evaluated with standard-ML and neural models.

The central question is therefore not simply which classifier scores highest. The question is: which

representation of the same vulnerable-to-fixed object exposes the most transferable vulnerability-related signal?

2. Related Motivation and Project Framing

This project is motivated by two concerns. First, strong performance in vulnerability detection does not always mean that a model has learned real, transferable vulnerability patterns. A model may score well because it memorizes repository-specific words, repeated fix templates, coding style, or other dataset shortcuts [1]. For this reason, this project reports both random-split and repository-disjoint results. In a random split, examples are divided randomly across train and test sets; in a repository-disjoint split, repositories used for testing are excluded from training. The random split shows easier in-dataset performance, while the repository-disjoint split asks a harder question: can the same representation still work when the test repositories were never seen during training?

Second, prior work supports treating code changes as security-relevant learning artifacts, not merely as metadata. PAVUDI studies patches as vulnerability-discovery units [2]; FuncVul focuses detection around edited code chunks [3]; VRepair learns from vulnerable and repaired code with diff context [4]; RepoSPD uses repository-level patch graphs [5]; and Robin uses patch semantic analysis for vulnerability verification [6]. Together, these works motivate this project’s core object:

`(func_before, func, diff_func).`

The contribution here is a controlled, lightweight comparison of representation layers for that object. Function text tests standalone local code signal; raw patch text tests edit signal; neutral structured diff features test explicit local change structure; and security-inspired features plus family motifs test whether targeted vulnerability-fix patterns add value. Therefore, the project does not claim patch-level representation is new or that it verifies patch correctness. Rather, it asks which local patch-level representations expose transferable vulnerability-fix signal before adding heavier AST, CFG, PDG, data-flow, graph, or static-analysis context.

3. Data and Learnable Object

The project uses MegaVul [7], a vulnerability dataset containing before/after function versions, function-level diffs, metadata, and optional Joern-derived graph files. To keep the study manageable and consistent, the project focuses on the C/C++ portion of the dataset. The important raw fields are `func`, `func_before`, `diff_func`, diff-line information, commit and file metadata, CWE/CVSS metadata, and repository identity. Repository identity is retained for repository-disjoint splitting and error analysis, not as a shortcut feature in the strict setting.

Table 1: Core fields that define one learnable function-pair object.

Field	Type	Role in this project
<code>func_before</code>	code text	Pre-fix or vulnerable-side function text used as the before side of the pair.
<code>func</code>	code text	Post-fix or fixed-side function text used as the after side of the pair.
<code>diff_func</code>	patch text	Local edit connecting the before and after versions; source for raw patch text and structured diff extraction.
<code>diff_line_info</code>	structured metadata	patch Used to derive added/deleted line counts, hunk counts, and patch-size features.
<code>commit_msg,</code> <code>file_path</code>	metadata text	Optional contextual metadata blocks.
<code>cwe_text,</code> <code>cvss_base_score</code>	label/severity data	meta- Used for family grouping, optional metadata features, and family-level analysis.
<code>repo_name</code>	grouping metadata	Used to construct repository-disjoint splits and analyze transfer.

The processed dataset normalizes missing text fields, parses diff-related fields into added/deleted summaries, retains repository identity for splitting, and groups CWE labels into broader vulnerability families. The output is a reproducible feature table that supports both standard-ML and neural ablation experiments.

In this study, one processed learning example is one patch-level function-pair object:

$$x_i = (\text{func_before}, \text{func}, \text{diff_func}).$$

This means that one dataset row is built from a pre-fix function, a post-fix function, and the local patch that connects the two versions.

Minimal function-pair object	
<code>func_before:</code>	<code>strcpy(dst, src);</code>
<code>func:</code>	<code>strncpy(dst, src, n);</code>
<code>diff_func:</code>	<code>- strcpy(dst, src);</code> <code>+ strncpy(dst, src, n);</code>

This example makes x_i concrete. The field `func_before` is the pre-fix version, `func` is the post-fix version, and `diff_func` records the local transformation between them. The learning problem is therefore not based on one standalone function alone. It is based on a before/after code-change object and the patch that links the two versions.

4. Methodology: Representation Design and Feature Extraction

The methodology is built around the idea that the same code-change pair can be encoded at different levels of interpretation. At the weakest level, the model consumes function or patch text directly. At a richer but still general level, the patch is converted into neutral structured change features. At a more specific level, security-inspired features represent common fix patterns such as added bounds checks, null checks, error returns, cleanup paths, or safer API changes. These features are motivated by patch-based vulnerability analysis and by MITRE CWE descriptions for improper input validation, out-of-bounds memory access, null-pointer errors, unchecked null returns, and improper handling of exceptional

conditions [2, 8, 9, 10, 11, 12]. At the highest engineered level, family-motif features encode recurring local patterns for input validation, memory/bounds, null-pointer, and simple locking/concurrency fixes; this design is motivated by work on reusable vulnerability repair templates and repair patterns [13, 14, 15].

Table 2: Experiment ladder and corresponding representation blocks.

Experiment setting	Representation added to the same function-pair object
Function only	Function text and simple code statistics.
Function + raw patch	Adds raw diff text and patch-size features.
Function + neutral structured patch	Adds vulnerability-agnostic local code-change structure.
Function + neutral structured patch + metadata	Adds commit, file-path, CWE, and severity metadata blocks.
Function + neutral structured patch + security-inspired features	Adds local vulnerability-fix indicators such as bounds/null checks and error returns.
Function + neutral structured patch + security-inspired features + metadata	Combines structured patch, security-inspired, and metadata blocks.
Function + neutral structured patch + family motifs	Adds family-specific local motif indicators.
Full structured local representation	Combines neutral patch, security-inspired, metadata, and family-motif blocks.

Table 3: Representative feature values extracted from one learnable patch-level function-pair object.

Experiment setting	Representative extracted feature	Example value
Function only	<code>code_line_count;</code> <code>code_kw_if;</code>	12; 1; 3
Function + raw patch	<code>code_pointer_count</code> <code>raw_diff_num_added;</code> <code>raw_diff_num_deleted;</code> <code>raw_diff_num_hunks</code>	4; 2; 1
Function + neutral structured patch	<code>neutral_added_kw_if;</code> <code>neutral_added_op_ge;</code> <code>neutral_added_memory_api</code>	1; 1; 1
Function + neutral structured patch + metadata	<code>commit_msg_text;</code> <code>file_path_text;</code> <code>cwe_text;</code> <code>cvss_base_score</code>	add validation; drivers/net/foo.c; memory bounds; 7.5
Function + neutral structured patch + security-inspired features	<code>sec_flag_added_bounds_check;</code> <code>sec_flag_added_error_return;</code> <code>sec_flag_safer_api_change</code>	1; 1; 1
Function + neutral structured patch + family motifs	<code>motif_mb_added_len_guard;</code> <code>motif_iv_added_validation_if;</code> <code>motif_iv_added_error_return_after_check</code>	1; 1; 1
Full structured local representation	<code>neutral_added_kw_if;</code> <code>sec_flag_added_bounds_check;</code> <code>commit_msg_text;</code> <code>motif_mb_added_len_guard</code>	1; 1; add validation; 1

Table 3 is sample-level, not dataset-level. It shows one processed row becoming one feature vector. Textual fields are vectorized with TF-IDF; numeric engineered fields are imputed and scaled; then the selected blocks are concatenated into the model input matrix.

5. Evaluation Design

The primary models are Logistic Regression and XGBoost. Logistic Regression tests whether the signal is mostly linear in sparse text and numeric features, while XGBoost tests nonlinear interactions among structured feature blocks. The neural extension evaluates an MLP fusion model and a transformer-based fusion model as comparison points rather than as the central claim.

Two splits are used. In the **random split**, examples are divided into train/validation/test sets with a 70/15/15 ratio, so examples from the same repository may appear in multiple splits. In the **repository-disjoint split**, repositories used in training are excluded from validation and testing. This split is the more important transfer test because it reduces repository-specific leakage and asks whether the learned representation generalizes to unseen repositories.

The main results report F1, PR-AUC, and ROC-AUC.

- **F1** measures the balance between precision and recall after one decision threshold is chosen. It is high only when the model’s positive predictions are mostly correct and the model also catches many of the true positive examples.
- **PR-AUC** measures the tradeoff between how often positive predictions are correct and how many true positive signals are caught. It uses precision versus recall. It is high when the model’s positive predictions remain mostly correct while still catching many of the true positive examples.
- **ROC-AUC** measures the tradeoff between catching true positive signals and mistakenly raising negative examples as positives. It uses recall/true positive rate versus false positive rate. It is high when the model catches many true positives while keeping false positives low.

These metrics are useful, especially PR-AUC, because this experiment evaluates whether each model input can find likely vulnerability-related function-pair examples without producing too many false alarms. Accuracy measures only overall correctness at one threshold, while PR-AUC captures the tradeoff between finding true positives and keeping positive predictions reliable.¹

6. Results

6.1 Standard-ML Ladder

The standard-ML ladder is the core empirical result. The main body reports XGBoost because it is the strongest standard-ML baseline; the complete Logistic Regression and XGBoost tables remain in Appendix A. Tables 4 and 5 show the random and repository-disjoint XGBoost results, and Figures 2 and 3 visualize the corresponding PR-AUC ladder.

The important pattern is consistent across both splits: function text alone contains limited signal, raw patch text adds useful information, and neutral structured patch features produce the largest jump. Metadata, security-inspired indicators, and family motifs refine the signal, but they do not replace the central value of explicit local code-change structure.

¹Accuracy and PR-AUC answer different questions. Accuracy evaluates the model after one fixed decision threshold has been chosen: at that cutoff, how many final labels are correct? PR-AUC evaluates the same model across many possible thresholds: as the cutoff changes, how well does the model balance finding true positives and avoiding false alarms among predicted positives?

Table 4: Random-split XGBoost results by representation.

Code	Representation	F1	PR-AUC	ROC-AUC
E0	Function only	0.6390	0.6780	0.6820
E1	Function + raw patch	0.6535	0.7497	0.7343
E2	Function + neutral structured patch	0.6852	0.7938	0.7756
E3	Function + neutral structured patch + metadata	0.6871	0.7902	0.7732
E4	Function + neutral structured patch + security-inspired features	0.6812	0.7891	0.7715
E5	Function + neutral structured patch + security-inspired features + metadata	0.6752	0.7880	0.7681

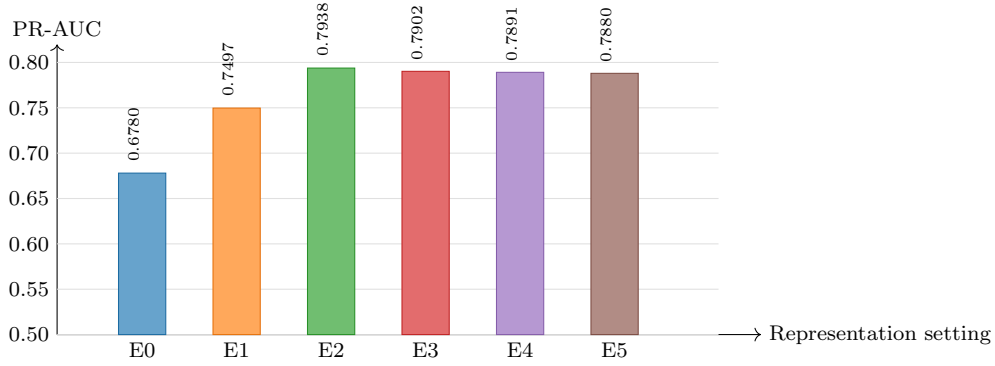


Figure 2: Random-split XGBoost PR-AUC across the representation ladder. The strongest gain comes from adding neutral structured patch features.

Table 5: Repository-disjoint XGBoost results by representation.

Code	Representation	F1	PR-AUC	ROC-AUC
E0	Function only	0.5396	0.5336	0.5395
E1	Function + raw patch	0.5575	0.6228	0.5970
E2	Function + neutral structured patch	0.5730	0.6649	0.6362
E3	Function + neutral structured patch + metadata	0.5779	0.6714	0.6420
E4	Function + neutral structured patch + security-inspired features	0.5794	0.6698	0.6434
E5	Function + neutral structured patch + security-inspired features + metadata	0.5833	0.6714	0.6424
E6	Function + neutral structured patch + family motifs	0.5892	0.6722	0.6460

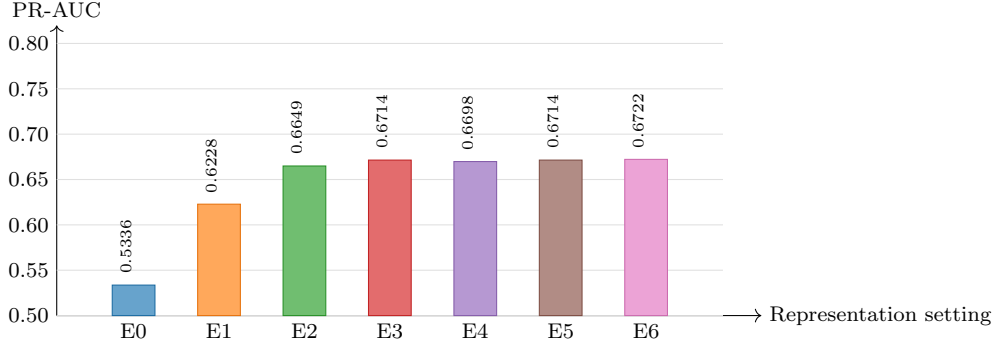


Figure 3: Repository-disjoint XGBoost PR-AUC across the representation ladder. The transfer setting is harder, but the same representation pattern remains visible.

In the random split, XGBoost improves from PR-AUC = 0.6780 with function-only features to 0.7938 after adding neutral structured patch features. In the repository-disjoint split, XGBoost improves from PR-AUC = 0.5336 with function-only features to 0.6649 with neutral structured patch features, then reaches 0.6722 when family motifs are added. The absolute scores drop under repository-disjoint evaluation, which is expected, but the representation trend is stable: the most important recoverable signal is not raw code alone; it is explicit local code-change structure.

6.2 Vulnerability-Family Analysis

Family-level results explain why one aggregate score is not enough. Some vulnerability families leave visible local patch evidence, while others depend on broader control flow, cleanup ordering, ownership conventions, or context outside the immediate function pair. Table 6 and Figure 4 summarize the repository-disjoint family results.

Table 6: Repository-disjoint standard-ML results by vulnerability family.

Vulnerability family	n	Best LR F1	Best XGB F1	Best XGB PR-AUC	Best setting
Input validation	1518	0.5383	0.5743	0.6784	N+S+M
Memory / bounds	2836	0.5516	0.6118	0.7111	N+S+M
Null pointer	676	0.5750	0.6405	0.7206	N+M
Locking / concurrency	228	0.5902	0.6122	0.6085	N+S+M
Resource / lifetime	1118	0.5473	0.5422	0.5002	Func
Other or mixed	3760	0.5494	0.5771	0.6621	N+M

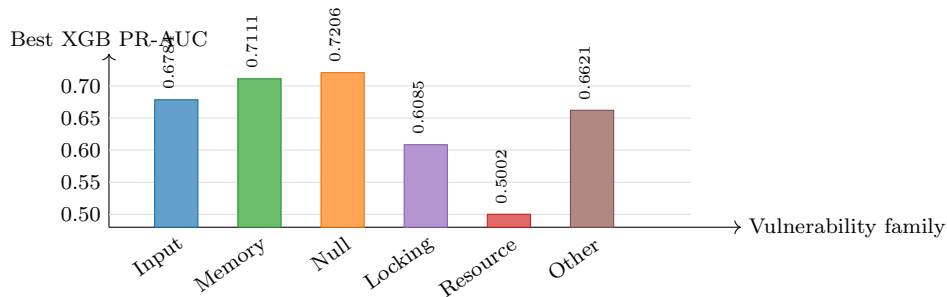


Figure 4: Repository-disjoint XGBoost PR-AUC by vulnerability family. Families with explicit local checks or API changes are more recoverable than resource/lifetime patterns.

The strongest family results occur for null-pointer, memory/bounds, and input-validation patterns because their fixes often introduce directly visible local syntax: null guards, range checks, validation conditions, error returns, and safer API substitutions. Resource/lifetime remains the weakest family because its fix evidence is often distributed across cleanup paths, object ownership, or longer control-flow context that the local function-pair representation only partially captures.

6.3 Family-Motif Analysis

Family motifs test whether recurring local fix templates add value beyond the neutral structured patch baseline. Table 7 and Figure 5 show that the effect is useful but selective: motif features improve several families, but they do not uniformly improve every family.

Table 7: Effect of adding family-motif features to the repository-disjoint XGBoost neutral-patch baseline.

Vulnerability family	n	Base F1	Motif F1	Δ F1	Base PR-AUC	Motif PR-AUC
Input validation	694	0.5536	0.5874	+0.0337	0.6614	0.6819
Locking / concurrency	236	0.6061	0.5570	-0.0491	0.6219	0.6652
Memory / bounds	2840	0.5905	0.6086	+0.0182	0.6992	0.7106
Null pointer	676	0.6259	0.6336	+0.0077	0.7054	0.7069
Other or mixed	4528	0.5679	0.5880	+0.0202	0.6599	0.6628
Resource / lifetime	1162	0.5307	0.5261	-0.0046	0.5719	0.5804

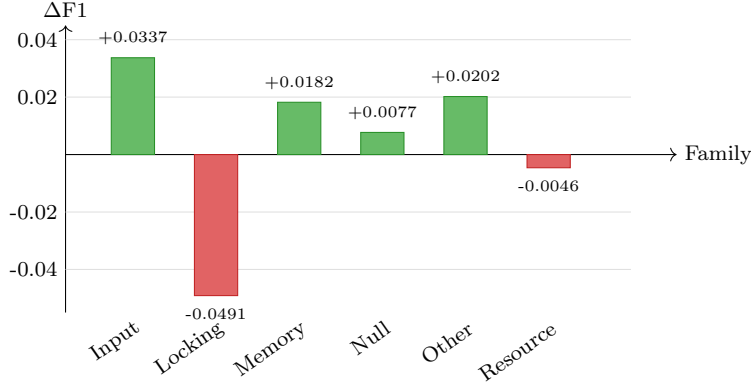


Figure 5: Change in F1 after adding family-motif features to the repository-disjoint XGBoost neutral-patch baseline. Positive bars indicate families where motifs add useful local signal.

Table 8: Overall repository-disjoint effect of adding family motifs.

Configuration	F1	PR-AUC	ROC-AUC
Neutral structured patch	0.5730	0.6649	0.6362
Neutral structured patch + family motifs	0.5892	0.6722	0.6460

The motif result should not be oversold. It is not a universal improvement layer. It is a selective refinement that helps when the feature library matches the local syntax of the fix. This supports the representation-level thesis of the paper: vulnerability signal is recoverable when the representation exposes the right local evidence.

6.4 Neural Extensions

The neural models were included to test whether learned dense representations improve beyond the strongest structured standard-ML baseline. Table 9 shows that they do not provide reliable incremental value in the current formulation.

Table 9: Repository-disjoint comparison of strongest standard-ML and neural results.

Model	Setting	Acc.	Prec.	Rec.	F1	PR-AUC	ROC-AUC
XGBoost	N + F	0.5914	0.5923	0.5860	0.5892	0.6722	0.6460
MLP	N	0.5152	0.5078	0.9854	0.6702	0.5880	0.5686
MLP	N + F	0.5129	0.5066	0.9882	0.6698	0.5857	0.5674
Transformer	N	0.5404	0.5538	0.4161	0.4752	0.5889	0.5680
Transformer	N + S + M + F	0.5393	0.5391	0.5418	0.5404	0.5798	0.5627

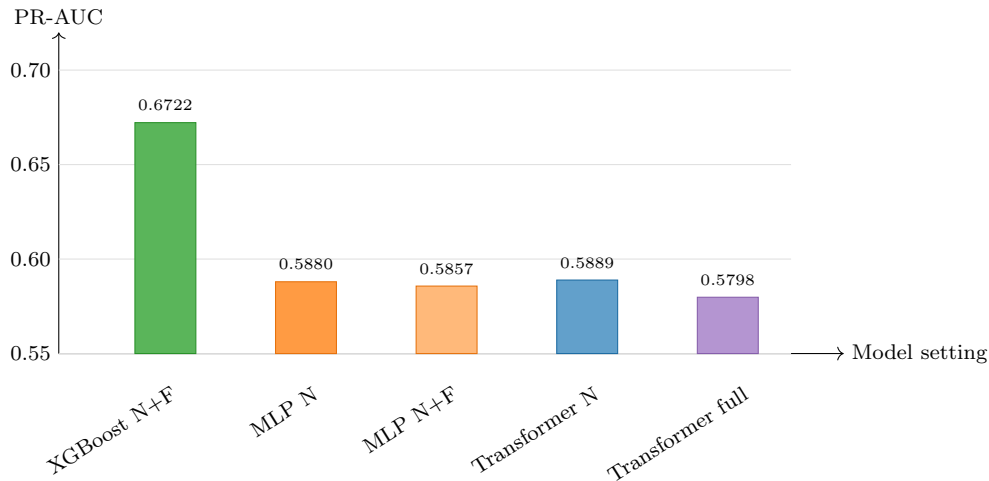


Figure 6: Repository-disjoint PR-AUC comparison between the strongest structured XGBoost result and the neural comparison models. The neural baselines remain below XGBoost on PR-AUC.

The MLP looks strong if we only look at F1, but this is misleading because it predicts almost everything as positive: it catches many true positives, but it also raises many false alarms. Its lower PR-AUC and ROC-AUC show that it does not separate positive and negative examples well. The transformer avoids this almost-all-positive behavior, but it still remains below the strongest structured XGBoost result on PR-AUC and ROC-AUC. The careful conclusion is not that neural models are ineffective for vulnerability analysis; it is that model complexity alone is not enough. These models likely need better-aligned inputs that represent the before/after edit relationship more directly.

7. Discussion

1. The results support one central interpretation: vulnerability-related signal is most recoverable when it appears as a visible local code change. Raw function text and raw diff text contain useful information, but the strongest gains come from structured local patch features that make the vulnerable-to-fixed transformation explicit. The neutral structured patch representation is therefore central because it organizes added guards, comparisons, return paths, memory-API changes, and size-related edits into model inputs that raw text alone does not expose cleanly.
2. The random and repository-disjoint results clarify what transfers. Random-split scores are higher because the model can benefit from shared repository vocabulary, repeated templates, and coding style. Repository-disjoint evaluation is harder because test repositories are unseen during training. The important result is that the same representation trend remains visible: structured patch features still improve over function-only and raw-patch inputs, although absolute scores are lower. This suggests that some local fix patterns, such as guards, bounds checks, null checks, error returns, and safer API substitutions, are partially transferable across repositories. The lower scores also limit the claim: the model recovers useful local patch structure for triage, not broad program-level vulnerability semantics.
3. The family-level results explain why the signal is uneven. Input-validation, memory/bounds, and null-pointer cases are more learnable because their fixes often leave concrete local evidence in the patch. Resource/lifetime cases are harder because the relevant evidence may depend on cleanup ordering, ownership transfer, exceptional control flow, or interprocedural context outside a single function-pair artifact. This also explains why family motifs help selectively: they work when the

motif matches the local syntax of the fix, but they cannot represent families whose evidence is distributed across broader program behavior.

4. The neural results should be interpreted carefully. The MLP has a high F1, but this is misleading because it predicts almost everything as positive: it catches many true positives, but also raises many false alarms. Its lower PR-AUC and ROC-AUC show that it does not separate positive and negative examples well. The transformer avoids this almost-all-positive behavior, but it still remains below the strongest structured XGBoost result on PR-AUC and ROC-AUC. The implication is not that neural models are ineffective for vulnerability analysis; it is that model complexity alone is not enough. These models likely need better-aligned inputs that represent the before/after edit relationship more directly. In this study, explicit structured patch features gave XGBoost a clearer local change signal than the tested MLP and Transformer inputs provided.

8. Limitations

This study is intentionally bounded. It uses local function-pair artifacts, so it cannot fully capture interprocedural behavior, repository-level semantics, ownership conventions, or resource-management ordering. The binary label also compresses many vulnerability mechanisms into one target. The family-motif library is hand-designed and selective, and the neural comparison does not redesign the models around pretrained code encoders, explicit edit alignment, cross-attention, or graph-context fusion. The project should therefore be read as a representation study and triage-oriented experiment, not as a complete vulnerability-verification system.

9. Future Work

Future work should move beyond simply adding more handcrafted motifs. Motif expansion remains useful for families with visible local fix patterns, but it is only one direction. A more important next step is to systematically investigate which representations work best with which model classes. Explicit structured patch features may remain strong for local edit patterns, code-language models may be better suited for before/after semantic reasoning, and graph or static-analysis representations may be necessary when the signal depends on control flow, data flow, resource lifetime, or interprocedural context. The long-term direction is therefore a hybrid architecture that combines multiple representations and routes or aggregates their signals, rather than relying on one monolithic model or one fixed feature design.

10. Team Contributions

This project was completed as a single-person project. Kiyoshi Watanabe handled project framing, data preparation, feature engineering, experiment implementation, ablation design, motif design, neural experiments, evaluation, interpretation, and report writing.

References

- [1] N. Risse, J. Liu, and M. Böhme, “Top Score on the Wrong Exam: On Benchmarking in Machine Learning for Vulnerability Detection,” *Proc. ACM Softw. Eng.*, vol. 2, no. ISSTA, pp. 1–23, 2025.
- [2] T. Ganz, E. Imgrund, M. Härterich, and K. Rieck, “PAVUDI: Patch-based Vulnerability Discovery using Machine Learning,” in *Proc. 39th Annual Computer Security Applications Conference*, 2023.

- [3] S. Halder, M. E. Ahmed, and S. Camtepe, “FuncVul: An Effective Function Level Vulnerability Detection Model using LLM and Code Chunk,” arXiv preprint arXiv:2506.19453, 2025.
- [4] Z. Chen, S. Kommrusch, and M. Monperrus, “Neural Transfer Learning for Repairing Security Vulnerabilities in C Code,” arXiv preprint arXiv:2104.08308, 2021.
- [5] X.-C. Wen, Z. Lin, and others, “Repository-Level Graph Representation Learning for Enhanced Security Patch Detection,” arXiv preprint arXiv:2412.08068, 2024.
- [6] S. Yang, Z. Xu, Y. Xiao, Z. Lang, W. Tang, Y. Liu, Z. Shi, H. Li, and L. Sun, “Towards Practical Binary Code Similarity Detection: Vulnerability Verification via Patch Semantic Analysis,” *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 6, Article 158, 2023. doi: 10.1145/3604608.
- [7] Icyrockton, “MegaVul,” GitHub repository. <https://github.com/icYROCKTON/megavul>
- [8] MITRE, “CWE-20: Improper Input Validation,” Common Weakness Enumeration. <https://cwe.mitre.org/data/definitions/20.html>
- [9] MITRE, “CWE-787: Out-of-bounds Write,” Common Weakness Enumeration. <https://cwe.mitre.org/data/definitions/787.html>
- [10] MITRE, “CWE-476: NULL Pointer Dereference,” Common Weakness Enumeration. <https://cwe.mitre.org/data/definitions/476.html>
- [11] MITRE, “CWE-690: Unchecked Return Value to NULL Pointer Dereference,” Common Weakness Enumeration. <https://cwe.mitre.org/data/definitions/690.html>
- [12] MITRE, “CWE-754: Improper Check for Unusual or Exceptional Conditions,” Common Weakness Enumeration. <https://cwe.mitre.org/data/definitions/754.html>
- [13] Q.-C. Bui et al., “VULTERMINATOR: Bringing Back Template-Based Vulnerability Repair,” 2025.
- [14] X. Li et al., “What Do They Fix? LLM-Aided Categorization of Vulnerability Repair Patterns,” NDSS, 2026.
- [15] Q.-C. Bui, R. Paramitha, D.-L. Vu, F. Massacci, and R. Scandariato, “APR4Vul: an empirical study of automatic program repair techniques on real-world Java vulnerabilities,” *Empirical Software Engineering*, vol. 29, no. 1, 2024.

A. Full Experiment Tables

Table 10: Complete standard-ML experiment ladder under the random split.

Experiment	Logistic Regression			XGBoost		
	F1	PR-AUC	ROC-AUC	F1	PR-AUC	ROC-AUC
Function only	0.5712	0.5835	0.5886	0.6390	0.6780	0.6820
Function + raw patch	0.5757	0.5966	0.5982	0.6535	0.7497	0.7343
Function + neutral structured patch	0.5732	0.5965	0.5982	0.6852	0.7938	0.7756
Function + neutral structured patch + metadata	0.5857	0.5979	0.6050	0.6871	0.7902	0.7732
Function + neutral structured patch + security-inspired features	0.5728	0.5967	0.5981	0.6812	0.7891	0.7715
Function + neutral structured patch + security-inspired features + metadata	0.5846	0.5980	0.6049	0.6752	0.7880	0.7681

Table 11: Complete standard-ML experiment ladder under the repository-disjoint split.

Experiment	Logistic Regression			XGBoost		
	F1	PR-AUC	ROC-AUC	F1	PR-AUC	ROC-AUC
Function only	0.5359	0.5320	0.5392	0.5396	0.5336	0.5395
Function + raw patch	0.5415	0.5423	0.5441	0.5575	0.6228	0.5970
Function + neutral structured patch	0.5385	0.5425	0.5442	0.5730	0.6649	0.6362
Function + neutral structured patch + metadata	0.5451	0.5411	0.5462	0.5779	0.6714	0.6420
Function + neutral structured patch + security-inspired features	0.5408	0.5422	0.5440	0.5794	0.6698	0.6434
Function + neutral structured patch + security-inspired features + metadata	0.5450	0.5409	0.5460	0.5833	0.6714	0.6424